MC MOBILECODE

MobileCode: A Phone-Native Harness for Verifiable Agentic Software Work

Phone-native control. Typed execution. Evidence-bound claims.

Harzva MobileCode Project

Technical Report v0 - Evidence snapshot: 2026-07-13

Repository: https://github.com/Harzva/mobilecode

STATUS	Advanced Beta candidate - physical-device closure pending
PRIMARY TARGET	Android and iOS control plane; Android Dev Harness execution
EVIDENCE DATE	2026-07-13

Abstract

AI coding agents are normally operated from desktop shells, remote integrated development environments, or cloud sandboxes. This leaves a gap on the device that is often closest to the user: the phone. A phone can display a chat interface, but a credible phone-native agent also needs to own execution state, permission boundaries, artifacts, runtime selection, verification, recovery, and release evidence. MobileCode treats the phone as this control plane. Models may be remote and heavyweight builds may run elsewhere, while the agent loop, typed actions, approval decisions, local files, previews, runtime diagnostics, and evidence records remain visible in the mobile application.

This report presents the current MobileCode architecture as a verifiable mobile harness rather than a remote-IDE skin. The system separates a Flutter control plane, a capability-aware runtime plane, and an evidence plane. RuntimeProvider exposes multiple backends, including a built-in Android Helper, an Alpine Linux sandbox, an external Termux fallback, a bounded embedded-lite path, cloud placeholders, and WebView-only degradation. ActionRunner accepts structured actions, blocks undeclared shell payloads, requests approval for mutation, and emits redacted ActionEvidence records. A CLI Hub catalog describes install, probe, authentication, read-only, and mutation tasks as typed contracts. Native WebView/WKWebView rendering handles low-cost PNG and PDF output, while an approved HyperFrames task provides local MP4 rendering when Node.js, Chromium, and FFmpeg are present.

The evidence snapshot records a July 13 full Flutter regression with 502 passing tests, a static-analysis result with zero errors and documented warning debt, and an Android emulator closeout for editable HTML, PNG/PDF export, HyperFrames lint/check/render inside an Alpine/PRoot guest, and authenticated Helper startup. The remote Helper 401 was traced to CI process scope: the emulator action executed adjacent script lines in separate shells, so a header stored in one line's local variable was absent from later curl requests even though Android had received the expected 15-character synthetic token. The workflow now sends the header explicitly in every request. Related hardening adds a shell-safe intent alias, a volatile per-request service credential snapshot, presence-only logging, explicit non-sticky startup, and bounded retries. Physical-device CLI login and business-task evidence remain incomplete, and provider-supported subscription quota refresh is not yet available. MobileCode is therefore presented as an advanced Beta candidate with a concrete release gate, not as a finished production system.

1. Introduction

The practical unit of an AI agent is not a model response. It is an execution trajectory containing observations, state transitions, tool calls, user approvals, runtime effects, verification events, artifacts, and recovery decisions. The execution harness turns latent model capability into behavior that can be inspected and controlled. On a desktop, the shell and file system provide much of this substrate implicitly. On a phone, those assumptions fail: background execution is constrained, app sandboxes are strict, local toolchains are incomplete, file access is mediated by platform APIs, and permission or battery policy can interrupt work.

MobileCode starts from a different decomposition:

```
Mobile agent = model capability + phone-native harness + selected runtime + verification evidence.
```

The model is one supplier of decisions. The mobile harness owns the interaction contract. The selected runtime provides only the capabilities it can truthfully expose. The verifier and evidence layer determine what can be claimed after execution. This decomposition avoids two common errors. First, a cloud model does not make the phone a remote IDE; the phone remains the owner of user intent, approval, state, files, and artifacts. Second, an Alpine or Termux-like environment does not automatically make heavyweight local inference possible; it is a tooling substrate whose package, ABI, memory, browser, and media limits remain explicit.

The report is organized around four questions:

- What must remain on the phone for MobileCode to be meaningfully phone-native?
- How can heterogeneous runtimes be selected without overstating capability?
- How can an agent execute useful work without accepting arbitrary model-provided shell strings?
- What evidence is required before a feature or release can be called complete?

The presentation style is informed by the systems structure of the OPENSQUILLA Agentic Routing report, especially its separation of problem definition, architecture, evaluation, limitations, and future work. MobileCode uses that structure for a different system problem: verifiable execution and artifact ownership on mobile devices.

2. Design Requirements

2.1 Phone-native ownership

The phone must own the user-visible control loop: conversation state, action previews, approvals, task progress, cancellation, recovery guidance, artifact browsing, and release status. Remote providers may supply models or heavy compute, but they cannot silently become the product's control plane.

2.2 Capability honesty

Every runtime reports capabilities and health before it is selected. Shell, Git, Node.js, Python, Flutter, Android build, package management, background execution, raw-shell permission, rootfs verification, network policy, and writable mounts are represented independently. A runtime is not considered ready merely because a process can be started.

2.3 Typed execution

High-level actions must be represented by a declared action name and typed payload. A model cannot supply an arbitrary `command`, `cmd`, or `shell` field to CLI Hub tasks. Mutation tasks require explicit approval. Read-only tasks receive output bounds and pagination controls. Authentication output is redacted before it reaches evidence or model context.

2.4 Recoverable state

Long-running tasks expose an identifier, status, logs, timestamps, exit code, duration, failure category, and cancellation semantics. The UI can reconstruct a task after a reconnect without interpreting the internal process model of each runtime.

2.5 Evidence-bound claims

Build success, emulator launch, physical-device behavior, third-party login, quota accuracy, and artifact integrity are separate claims. Each requires separate evidence. A screenshot proves a visible state; it does not prove hidden runtime behavior. A passing unit test proves a contract; it does not prove a provider login. A generated file proves output existence; a checksum and verifier establish identity and validity.

3. System Model

Let a user task be q, the mobile harness state at step t be h_t, the set of runtime providers be R, and the set of declared typed actions be A. The harness selects an action and runtime:

$$(a_t, r_t) = g(q, h_t, A, R)$$

subject to three gates:

1. $\mathrm{declared}(a_t)$: the action and payload are present in a local catalog or built-in schema;

2. `capable(r_t, a_t)`: the runtime health report satisfies the action's requirements;
3. `approved(a_t, h_t)`: a mutation or sensitive action has explicit user approval.

Execution produces a result and an evidence record:

```
e_t = (request_id, action, params_summary, runtime, start, end, status, logs, failure_kind, recovery, metadata, redaction)
```

The evidence record is the boundary between a runtime effect and a product claim. It deliberately separates pre-execution intent, the selected runtime, the realized effect, and the verifier outcome. This makes failures useful: a missing dependency, blocked command, timeout, authentication failure, lost runtime, or approval requirement becomes a typed recovery state rather than an unstructured error string.

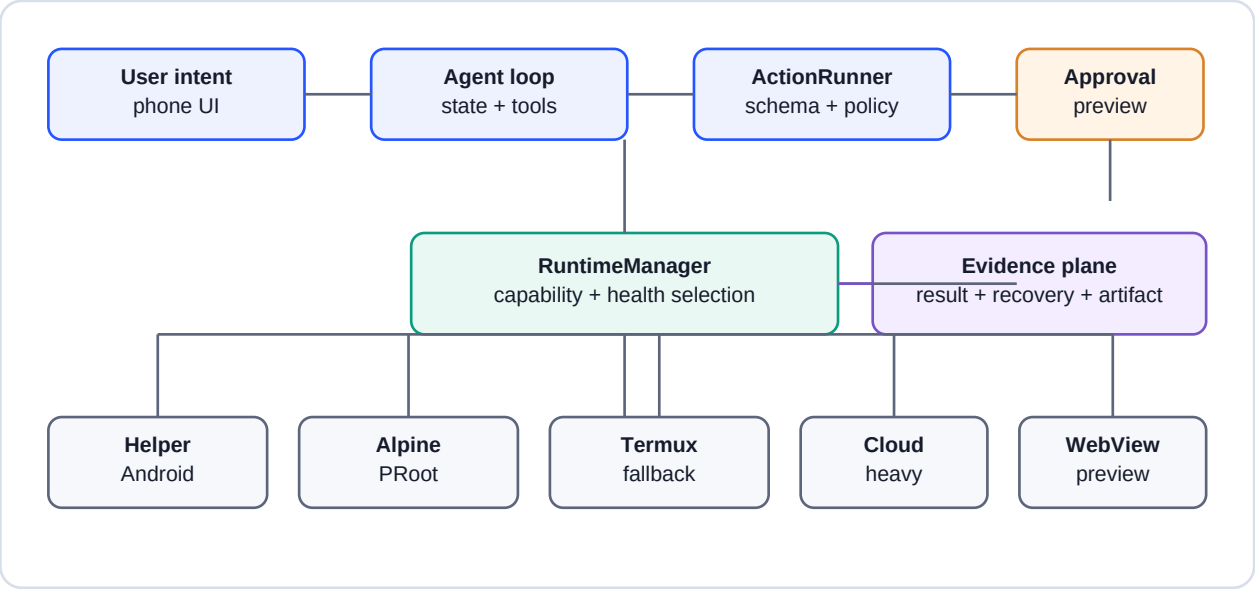


Figure 1. MobileCode's three-plane architecture. The phone owns the control and evidence planes. Execution can remain local, use an external mobile runtime, or delegate a heavy build to a remote service while preserving the same typed contract.

4. Architecture

4.1 Control plane

The Flutter application provides the conversation, file workspace, settings, capability center, extension center, build preview, subscription usage surface, task trace, recovery cards, and artifact preview. AgentLoopController converts model tool calls into structured MobileCode actions. ActionRunner validates the action and produces ActionEvidence. Approval and progress are visible in the same trace that initiated the task.

The control plane intentionally contains product policy. It decides which actions are exposed to the model, which require approval, which runtime family is eligible, how much output may return, and which recovery action the user should see. These decisions do not belong inside a provider-specific shell script.

4.2 Runtime plane

RuntimeProvider is the common contract. It exposes initialization, capability reporting, health, command execution, streaming, workspace synchronization, web/APK build requests, install/launch operations, and task cancellation. Optional interfaces add task monitoring and typed task execution.

The current provider order is:

Provider	Intended role	Current boundary
MobileCode Helper	Built-in Android service for app-owned execution	Localhost service, authenticated, bounded by app sandbox
Linux Sandbox	Alpine/PRoot CLI substrate	Typed tasks and profiles; not a generic model inference engine
Termux daemon / External Termux	Advanced or development fallback	External dependency and separate lifecycle
Embedded Lite	Controlled preflight and preview metadata	No generic shell, package ecosystem, or heavyweight build
Cloud Runtime	Future heavy-task delegation	Explicit placeholder, not silently selected as complete
WebView Only	Graceful preview fallback	No shell or project toolchain claims

RuntimeManager initializes providers, records health, selects the first ready provider for general execution, and can explicitly route CLI Hub tasks to the Linux sandbox. Selection is therefore capability-aware and observable. A failed health check becomes a RuntimeHealth result with missing dependencies and recovery actions.

4.3 Evidence plane

ActionEvidence unifies runtime, Git, collaboration, and release observations. The record includes timestamps, success, logs, failure category, recovery actions, changed files, metadata, and a redaction flag. Evidence is stored separately from the UI widgets that render it. This permits the same record to support task recovery, approval history, diagnostics, release QA, and future benchmark traces.

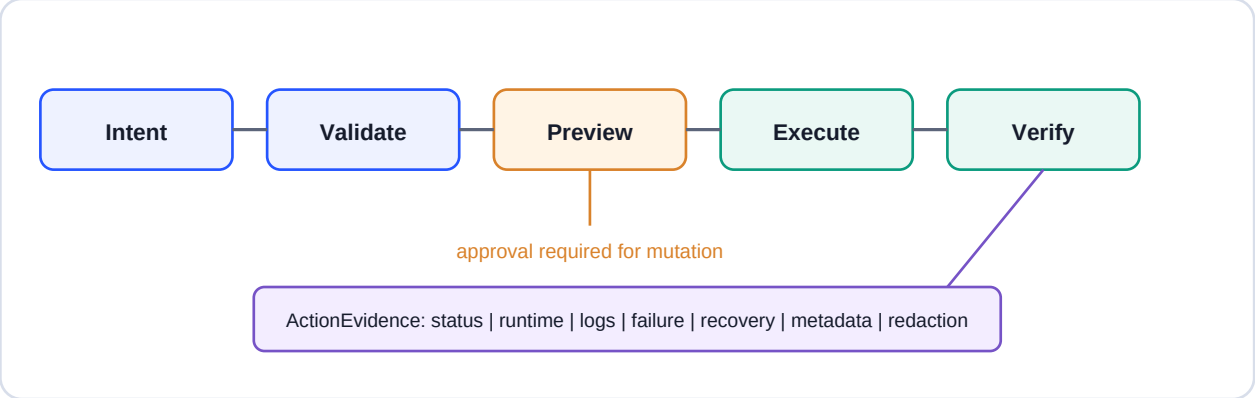


Figure 2. Preview-first action lifecycle. A mutation is not started until the approved replay is received. Runtime output is normalized and redacted before it becomes evidence or model context.

5. Typed Actions and CLI Hub

CLI Hub is a local catalog and schema for bounded command-line capabilities. An entry declares identity, source, support level, risk, credential policy, install profile, health probe, authentication tasks, and business tasks. The schema rejects duplicate identifiers, missing safety fields, undeclared raw shell, and credential-like payload fields.

The first execution slice supports four classes of behavior:

- Probe: inspect a version or runtime state without mutation.
- Authentication: start or inspect an official login flow while redacting codes and tokens.
- Read-only business task: list repositories, files, messages, or spaces with bounds and pagination.
- Mutation: install a package, render an artifact, or alter state only after approval.

For an approved task, the product path is:

`Chat intent -> ToolCallAdapter -> cli_hub_task -> ActionRunner -> catalog gate -> approval -> RuntimeManager -> LinuxSandboxRuntimeProvider -> native runner -> evidence.`

The task payload contains identifiers and bounded scalar data rather than a shell command. For example, HyperFrames rendering uses a task kind, a workspace-relative MP4 path, an optional composition identifier, and an approval bit. The native runner reconstructs the allowlisted command. Output paths containing parent traversal, absolute paths, unsupported extensions, or non-simple composition identifiers are rejected.

This design limits flexibility by intent. MobileCode prefers a smaller set of inspectable actions over a universal agent shell. Full access can exist as a separately governed expert mode, but it is not the default contract for a model-generated action.

6. Runtime Capability Routing

MobileCode runtime routing differs from model routing. It does not decide which language model should answer a prompt; it decides which execution substrate can satisfy a typed action while preserving policy. The router is currently deterministic and health-driven rather than learned.

For an action `a`, define the required capability vector `c(a)` and a runtime health vector `c(r)`. A runtime is eligible when `c(a)` is a subset of `c(r)` and policy constraints are satisfied. Among eligible runtimes, selection prefers product-native providers and degrades toward external or preview-only providers. The effective cost includes startup delay, dependency installation, network availability, battery impact, and recovery cost, not only CPU time.

This matters on mobile. A cold Alpine profile may take tens of minutes to install, while its verified fast path completes in under one second. A native WebView PNG export is therefore preferred over launching a Node/Chromium media stack. The heavier runtime is justified for MP4, where native WebView does not provide the required artifact.

The current runtime decision policy can be summarized as:

Work class	Preferred path	Escalation
File edit and preview	Flutter + native WebView/WKWebView	No runtime required
PNG/PDF export	Native renderer	Explicit failure if platform channel unavailable
Typed CLI probe	Linux sandbox installed profile	Helper or external runtime when declared
MP4 render	Approved HyperFrames profile	Host/Helper worker if mobile dependencies fail
APK or heavy build	GitHub Actions / capable external runtime	Never claim Embedded Lite support
Heavy VLM inference	Remote/provider-native model path	Alpine is not treated as inference enablement

7. Artifact Pipeline

MobileCode treats artifacts as first-class outputs. An HTML file remains editable source rather than a screenshot-only result. The native renderer can preview it and produce app-owned PNG or PDF files. The output record includes path, size, dimensions or page count when available, backend identity, and SHA-256.

For video, the HyperFrames CLI is exposed through typed tasks: probe, lint, check, compositions, and render. Render is a mutation because it writes an MP4 and can consume significant time and storage. The Android

Alpine runner pins the system Chromium path and software screenshot capture to avoid an unsuitable managed-browser download and lock path inside PRoot.

PPTX is a separate worker boundary. The current visual mode captures each HTML slide and places it as a full-slide image for fidelity. Editable-text mode is reserved but fails explicitly because a faithful general HTML-to-editable-PPTX conversion is not yet implemented. This is an example of capability honesty: unsupported editability is not represented as a successful visual export.

7.1 July 13 evidence

Evidence item	Result	Scope
Host HyperFrames 0.7.55 check	Passed	Lint, runtime, and layout gates on poster fixture
Host MP4 render	Passed	1920x1080, 10 seconds
Android Alpine profile install	Passed	4 GB API 36 arm64 emulator; Node 24.17, npm 11.12, Chromium 150, FFmpeg 8.1.2
Android HyperFrames lint/check/render	Passed	Alpine/PRoot; final focused run 60.572 seconds
Android MP4 artifact	Passed	H.264, 640x360, 30 fps, 1 second, 56,598 bytes
Native HTML PNG/PDF instrumented tests	2/2 passed	App-private artifacts
Editable HTML UI flow	Passed	Edit, save, reload, PNG share-sheet proof
Physical-device proof	Pending	Emulator evidence must not be relabeled as physical-device evidence

The package mutation log still contains a known Alpine `apk` database permission warning. Runtime postconditions verify the installed binaries, but the warning remains a release-quality defect until the underlying write path is clean.

8. Security and Privacy Model

MobileCode assumes model output, external catalogs, runtime stdout, authentication flows, workspace files, and remote provider responses can all contain untrusted or sensitive material.

The current controls include:

- Localhost authentication for the built-in Helper and desktop daemon.
- Typed payload gates that reject shell and credential-like keys.
- Workspace-relative path validation for artifacts and task working directories.
- Explicit approval for install, login, mutation, render, commit, and other sensitive actions.
- Output bounds, pagination, timeout, and best-effort cancellation.
- Redaction of bearer tokens, OAuth codes, cookies, sessions, `.env` paths, email addresses, phone numbers, and local paths where required.
- Secure storage for provider credentials, separated from provider account metadata.
- Catalog source disclosure, SHA-256 integrity checks, Ed25519 signature verification, and trusted-key lifecycle structures.
- Pure/Dev Harness build separation so the Pure APK does not silently include Alpine rootfs assets.

These controls reduce but do not eliminate risk. A mobile app with broad file or accessibility permission remains sensitive. Local evidence can expose project names or task intent if redaction rules are incomplete. A signed extension can still be malicious if the signing key is compromised. A Helper token prevents accidental localhost access but does not replace Android sandboxing or user approval.

9. Evaluation Methodology

MobileCode uses layered evaluation because no single test establishes product readiness.

9.1 Contract tests

Flutter unit and widget tests cover action mapping, evidence serialization, approval behavior, runtime selection, CLI catalog validation, trusted keys, usage state, capability screens, recovery cards, and output redaction. Python and Kotlin focused tests cover helper-side typed tasks and native runners.

9.2 Static gates

Analyzer coverage is gradually expanded from a legacy quarantine. `git diff --check`, schema validation, changed-file secret scanning, and path/credential pattern checks serve as pre-commit gates. A passing analyzer with quarantined paths is recorded as such; it is not equivalent to complete historical code cleanup.

9.3 Build and emulator gates

Pure and Dev Harness APKs are built separately. Emulator QA installs and launches the APK, probes Helper health, exercises selected feature flows, captures UI hierarchy and screenshots, and scans logcat for crash signatures. Instrumented tests verify native channels and generated artifacts.

9.4 Remote and physical-device gates

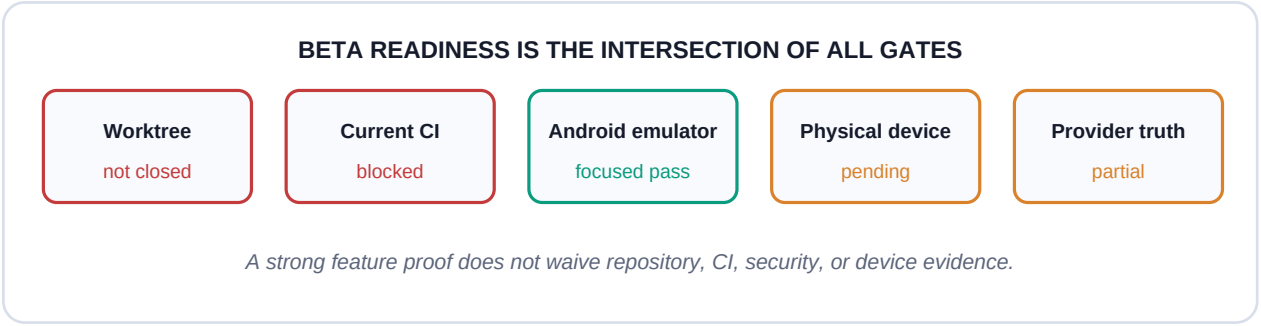
GitHub Actions is the repository-side source of truth for pull-request build and test status. Physical-device QA is a distinct gate for Android permissions, external apps, official login, background behavior, and performance. iOS requires simulator and physical-device evidence appropriate to Xcode signing and CoreDevice state.

9.5 Evidence snapshot

Date	Gate	Recorded result	Interpretation
2026-07-13	Full Flutter tests	502 passed	Current complete local suite, including July CLI Hub and artifact additions
2026-07-13	Flutter analyzer	0 errors, 241 warnings, 4,160 infos; exit 2	No semantic errors; strict-lint and legacy cleanup debt remains explicit
2026-07-13	Pure and Dev Harness APKs	Both current debug flavors built	Pure validates release-like boundaries; Dev Harness includes packaged Alpine assets
2026-07-13	HyperFrames/HTML focused QA	Passed current slice	Native PNG/PDF and local MP4 evidence complement the full Flutter regression
2026-07-13	Authenticated Android smoke	Passed locally on current Pure APK	Correct token returns 200, wrong token returns 401, typed `pwd` and app launch succeed
2026-07-13	Remote PR Actions	Passed on feature code head `d567337`	Mobile Runtime CI run 29268282842 and Android App Smoke run 29268282931 passed; the latter completed in 15m56s
2026-07-13	Android native artifact tests	2 of 2 passed on Dev Harness	Instrumented WebView PNG and PDF generation passed on API 36 emulator
2026-07-13	iOS simulator launch	Passed on iPhone 17 Pro simulator	Signed simulator build launched with no fatal Runner log signatures
2026-07-13	Physical iPhone install	Blocked by locked device	Paired iPhone 11 was visible, but CoreDevice tunnel was disconnected and install returned lock errors 10003/1016

10. Beta Release Gate

The roadmap contains 28 task documents. At this snapshot, 26 are closed or accepted and two remain in progress: T26 Subscription Login and Usage Hub, and T27 Chat CLI Hub Agent Tool Calling. This ratio is useful for planning but is not itself a release percentage; several closed roadmap items intentionally preserve deferred production work.



The Beta gate requires all of the following:

1. Repository convergence: the feature branch, pull request, roadmap, generated report, and evidence references describe the same commit.
2. Clean working tree: generated files and private local state are ignored; source changes are split into reviewable commits; secret scanning and `git diff --check` pass.
3. Remote CI: Flutter/runtime checks, Android build, and Android emulator smoke pass on the current head. The Helper 401 failure must be fixed rather than waived.
4. Android device evidence: install, launch, Helper health, Linux sandbox profile, CLI login recovery, one read-only business task, artifact export, and background/permission behavior are recorded on a physical device when available.
5. iOS evidence: simulator build and launch pass; physical-device evidence is captured when signing and a connected device are available. Unsupported Linux-sandbox behavior on iOS remains explicit.
6. Provider truthfulness: manual API-key flows remain separate from consumer subscription login; quota cards remain labeled mock until an official source is verified.
7. Known limitations: Alpine package warning, catalog trust-chain gaps, editable PPTX absence, and deferred marketplace behavior remain documented.

11. Limitations and Threats to Validity

First, most July execution evidence comes from an Android emulator rather than a physical phone. Emulator filesystem, networking, memory pressure, battery policy, and vendor behavior differ from real devices. Second, the 502-test regression covers the Flutter layer; focused Android compile, service, and instrumented tests provide the native-layer evidence. These layers are complementary rather than interchangeable. Third, several runtime capabilities depend on downloaded packages and public mirrors whose availability can vary. The July run used a local mirror cache to avoid transfer stalls; that cache is not shipped in the APK.

Fourth, typed actions limit injection risk but do not prove semantic safety. A declared task can still cause undesirable effects if its implementation or approval copy is wrong. Fifth, extension signature verification depends on key distribution, rotation, and revocation governance that is not fully closed. Sixth, subscription providers expose different official authentication and quota boundaries; a manual API key validation does not imply consumer subscription reuse. Finally, roadmap completion is not a controlled benchmark and should not be treated as a product-quality score.

12. Future Work

The immediate work after repository convergence is physical-device closure: unlock and reconnect the paired iPhone for install/launch evidence, attach an Android handset for permission and background checks, and capture real CLI login plus one bounded business task. T26 should then complete only the provider-supported login and quota paths that can be implemented without cookie scraping or private session import. T27 should close true Android CLI login and business-task flows, marketplace persistence, and the remaining catalog trust chain.

Longer-term research can build on the evidence plane. Every typed action already produces state, runtime choice, outcome, recovery, duration, and artifact metadata. With privacy-preserving governance, these records can support capability routing, failure clustering, verifier improvement, and MobileHarnessBench evaluation. The goal is not to collect raw user logs. The useful corpus is a structured, redacted set of execution outcomes whose provenance and consent are explicit.

13. Conclusion

MobileCode demonstrates that phone-native agentic software work is not primarily a problem of fitting a desktop toolchain into a small screen. It is a harness problem. The phone must own control, approval, state, artifacts, and evidence while selecting the smallest runtime that can truthfully perform the next action. Typed tasks, capability-aware runtime routing, native artifact paths, and redacted evidence provide a practical architecture for that goal.

The current implementation has crossed the prototype boundary: it contains real runtime abstractions, native Android execution, Alpine/PRoot CLI profiles, approval and evidence flows, editable artifact output, and focused on-device media rendering. Repository cleanup, split commits, pull-request synchronization, local Helper authentication, the Pure APK build, and remote PR Actions now have current evidence. It has not crossed the release boundary because provider flows and physical-device CLI, permission, background, and install evidence remain incomplete. Treating those blockers as first-class evidence is part of the architecture, not an admission external to it.

Appendix A. Reproducibility Map

Claim	Primary repository evidence
Runtime provider contract	`mobile_agent/lib/services/runtime_provider.dart`
Runtime selection	`mobile_agent/lib/services/runtime_manager.dart`
Typed action and evidence gate	`mobile_agent/lib/core/evidence/action_runner.dart`
Evidence schema	`mobile_agent/lib/core/evidence/evidence_model.dart`
CLI catalog and trust schema	`cli-hub/` and `mobile_agent/lib/services/cli_hub_catalog_service.dart`
Linux sandbox implementation	`mobile_agent/lib/services/linux_sandbox_provider.dart` and native runner
Native HTML rendering	`mobile_agent/lib/services/html_render_provider.dart`, Android/iOS runners
HyperFrames boundary	`docs/hyperframes-cli-compatibility.md`
July Android evidence	`mobile_agent/qa-output/android-hyperframes-closeout-20260713/RESULT.md`
Roadmap state	`roadmp.md`, `roadmap/tasks/T26-*`, `roadmap/tasks/T27-*`
Remote CI	`.github/workflows/mobile-runtime-ci.yml`, `.github/workflows/android-app-test.yml`

References

1 TokenRhythm Technologies. Agentic Routing: The Harness-Native Data Flywheel. OPENSQUILLA technical report, 2026. https://github.com/opensquilla/opensquilla/blob/main/docs/releases/agentic_routing_v0.pdf

2 Harzva. MobileCode repository. https://github.com/Harzva/mobilecode

3 Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? 2023.

4 Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

5 Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

6 Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023.

7 Android Developers. Services, app sandbox, storage, and background execution documentation. https://developer.android.com/

8 Flutter. Platform channels and application development documentation. https://docs.flutter.dev/

9 HyperFrames. HTML-first video framework and CLI. https://github.com/heygen-com/hyperframes

10 GitHub Docs. GitHub Actions, OAuth, and REST API documentation. https://docs.github.com/